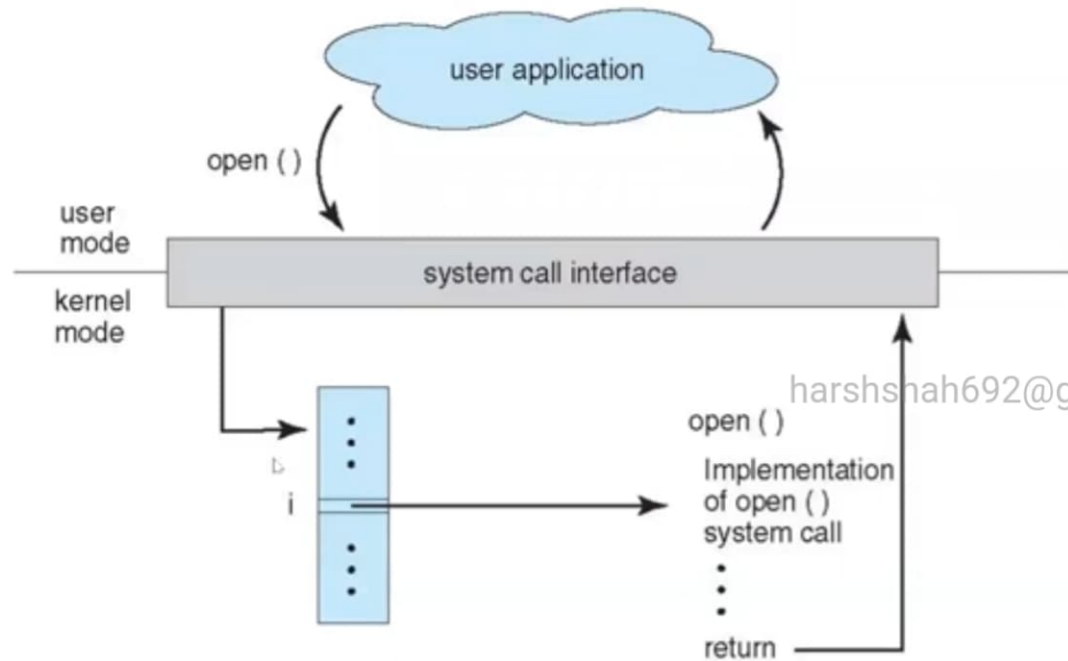


# System Call Implementation



- Typically, a number associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

# API – System Call – OS Relationship



harshshah692@gmail.com

# System Call Parameter Passing



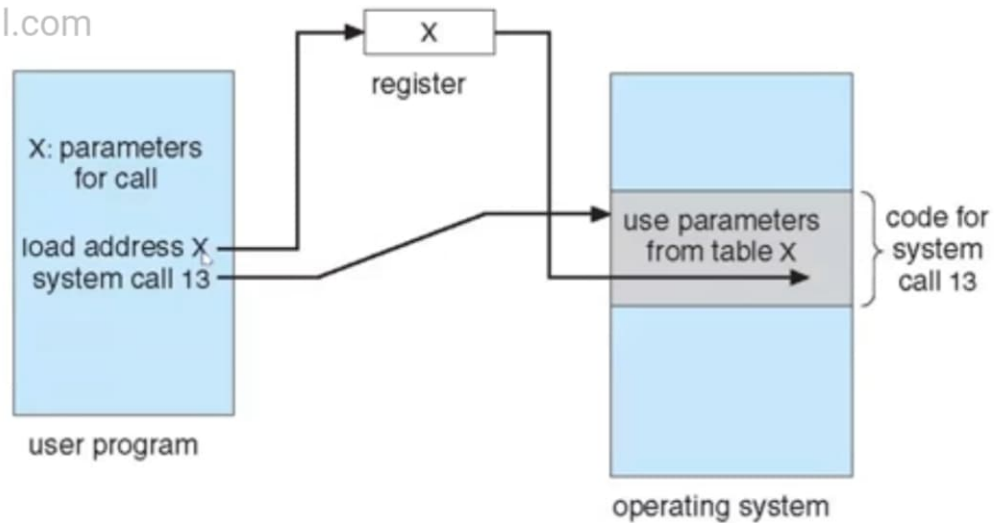
- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in registers
    - In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

harshshah692@gmail.com

# Parameter Passing via Table



harshshah692@gmail.com





- Process control
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes

# Types of System Calls



- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

harshshan692@gmail.com



-32:48

1x

auto



# Types of System Calls (Cont.)



- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

harshshah692@gmail.com





# Examples of Unix and Windows System Calls



|                         | Windows                                                                             | Unix                                   |
|-------------------------|-------------------------------------------------------------------------------------|----------------------------------------|
| Process Control         | CreateProcess()<br>ExitProcess()<br>WaitForSingleObject()                           | fork()<br>exit()<br>wait()             |
| File Manipulation       | CreateFile()<br>ReadFile()<br>WriteFile()<br>CloseHandle()                          | open()<br>read()<br>write()<br>close() |
| Device Manipulation     | SetConsoleMode()<br>ReadConsole()<br>WriteConsole()                                 | ioctl()<br>read()<br>write()           |
| Information Maintenance | GetCurrentProcessID()<br>SetTimer()<br>Sleep()                                      | getpid()<br>alarm()<br>sleep()         |
| Communication           | CreatePipe()<br>CreateFileMapping()<br>MapViewOfFile()                              | pipe()<br>shmget()<br>mmap()           |
| Protection              | SetFileSecurity()<br>InitializeSecurityDescriptor()<br>SetSecurityDescriptorGroup() | chmod()<br>umask()<br>chown()          |

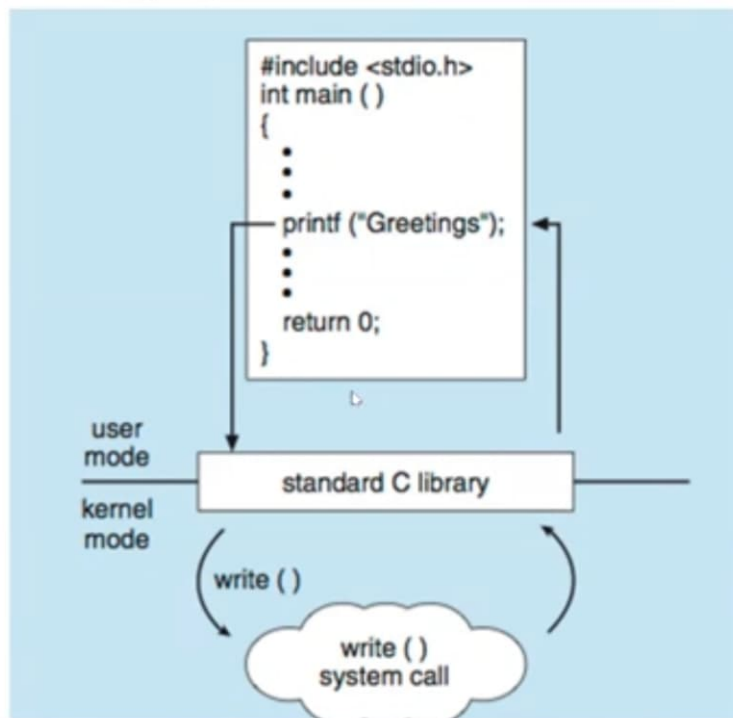
harshshah692@gmail.com





# Standard C Library Example

- C program invoking printf() library call, which calls write() system call





```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```

harshshah692@gmail.com



```
#include <stdio.h>
#include <windows.h>
```

```
int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
```

```
    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
```

```
    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si,
        &pi))
```

```
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
```

```
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");
```

```
    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

harshshah692@gmail.com

ACE Live Class 8



## Common and well known system calls are:

harshshah692@gmail.com



- access: checks if calling process has file access
- alarm: sets a process's alarm clock
- chdir: changes the working directory
- chmod: changes the mode of a file
- chown: changes the ownership of a file
- chroot: changes the root directory
- close: closes a file descriptor
- dup, dup2: duplicates an open file descriptor
- execl, execv, execl, execve, execlp, execvp: executes a file
- exit: exits a process
- fcntl: controls open files
- fork: creates a new process
- getpid, getpgrp, getppid: gets group and process IDs
- getuid, geteuid, getgid, getegid: gets user and group IDs
- ioctl: controls character devices
- kill: sends a signal to one or more processes
- link: links a new file name to an existing file



- time: number of seconds since 1/1/1970
- times: gets process and child process times
- ulimit: gets and sets user limits
- umask: gets and sets file creation mask
- umount: unmounts a file system
- uname: gets system information
- unlink: removes directory entry
- ustat: gets file system statistics
- utime: sets file access and modification times
- wait: waits for a child process to stop or terminate

harshshah692@gmail.com

